

Primes, Sieves, and Modular Arithmetic

Algorithmic Reasoning, Complexity, and Precomputation

NT-ops

Phase 1: Structural Primality & Complexity

- Bounding the mathematical search space.
- $\mathcal{O}(\sqrt{N})$ Prime Factorization Analysis.

Lecture Roadmap

Phase 1: Structural Primality & Complexity

- Bounding the mathematical search space.
- $\mathcal{O}(\sqrt{N})$ Prime Factorization Analysis.

Phase 2: Bulk Precomputation (The Sieve)

- The Sieve of Eratosthenes & Harmonic Primes.
- Strict Memory Analysis: Surviving 256MB limits.
- Smallest Prime Factor (SPF) for $\mathcal{O}(\log N)$ queries.

Lecture Roadmap

Phase 1: Structural Primality & Complexity

- Bounding the mathematical search space.
- $\mathcal{O}(\sqrt{N})$ Prime Factorization Analysis.

Phase 2: Bulk Precomputation (The Sieve)

- The Sieve of Eratosthenes & Harmonic Primes.
- Strict Memory Analysis: Surviving 256MB limits.
- Smallest Prime Factor (SPF) for $\mathcal{O}(\log N)$ queries.

Phase 3: Advanced Modular Arithmetic

- The Modulo Division Failure.
- Fermat's Little Theorem & Inverse Computation.
- Euler's Totient Function (ETF) for composite modulus.

Primality

Definition

An integer $N \in \mathbb{Z}^+$ is prime ($N \in \mathbb{P}$) **if and only if** $N > 1$ and its only positive divisors are 1 and N .

Primality

Definition

An integer $N \in \mathbb{Z}^+$ is prime ($N \in \mathbb{P}$) **if and only if** $N > 1$ and its only positive divisors are 1 and N .

How to find whether a given number is prime or not ?

Primality

Definition

An integer $N \in \mathbb{Z}^+$ is prime ($N \in \mathbb{P}$) **if and only if** $N > 1$ and its only positive divisors are 1 and N .

How to find whether a given number is prime or not ?

The Naive Approach: To verify if N is prime, we loop from 2 to $N - 1$.

Primality

Definition

An integer $N \in \mathbb{Z}^+$ is prime ($N \in \mathbb{P}$) **if and only if** $N > 1$ and its only positive divisors are 1 and N .

How to find whether a given number is prime or not ?

The Naive Approach: To verify if N is prime, we loop from 2 to $N - 1$.

Question: The Computational Limit

In CP, we process $\approx 10^8$ operations per second. If a problem provides $N = 10^{12}$, our naive $\mathcal{O}(N)$ verification will trigger a Time Limit Exceeded (TLE).

How can we mathematically prove that we do not need to check all the way to $N - 1$?

Bounding the Search Space

Let us assume N is composite. Therefore, it can be factored into $N = a \cdot b$, where $1 < a \leq b < N$.

Bounding the Search Space

Let us assume N is composite. Therefore, it can be factored into $N = a \cdot b$, where $1 < a \leq b < N$.

Consider the Edge Case: What happens if we assume *both* factors a and b are strictly greater than $\sqrt{N}$?

Bounding the Search Space

Let us assume N is composite. Therefore, it can be factored into $N = a \cdot b$, where $1 < a \leq b < N$.

Consider the Edge Case: What happens if we assume *both* factors a and b are strictly greater than $\sqrt{N}$?

The Contradiction:

- If $a > \sqrt{N}$ and $b > \sqrt{N}$...
- Multiplying them yields: $a \cdot b > \sqrt{N} \cdot \sqrt{N}$
- Therefore, $N > N$. (Mathematical Contradiction $\perp$)

Bounding the Search Space

Let us assume N is composite. Therefore, it can be factored into $N = a \cdot b$, where $1 < a \leq b < N$.

Consider the Edge Case: What happens if we assume *both* factors a and b are strictly greater than $\sqrt{N}$?

The Contradiction:

- If $a > \sqrt{N}$ and $b > \sqrt{N}$...
- Multiplying them yields: $a \cdot b > \sqrt{N} \cdot \sqrt{N}$
- Therefore, $N > N$. (Mathematical Contradiction $\perp$)

Theorem (The Square Root Bound)

If N is composite, it must possess at least one prime factor p such that $p \leq \sqrt{N}$. We only need to check divisors up to $\sqrt{N}$.

Prime Factorization

Question: The Residual Factor

If we consecutively divide N by all its prime factors up to $\sqrt{N}$, and the remaining value is strictly greater than 1, what does that remaining value represent?

Prime Factorization

Question: The Residual Factor

If we consecutively divide N by all its prime factors up to $\sqrt{N}$, and the remaining value is strictly greater than 1, what does that remaining value represent?

Corollary: An integer $N > 1$ can possess *at most one* prime factor strictly greater than $\sqrt{N}$.

Prime Factorization

Question: The Residual Factor

If we consecutively divide N by all its prime factors up to $\sqrt{N}$, and the remaining value is strictly greater than 1, what does that remaining value represent?

Corollary: An integer $N > 1$ can possess *at most one* prime factor strictly greater than $\sqrt{N}$.

Listing 3: Optimized Factorization

```
std::vector<long long> primeFactors(long long n) {
    std::vector<long long> factors;
    for (long long i = 2; i * i <= n; i++) {
        while (n % i == 0) {
            factors.push_back(i);
            n /= i;
        }
    }
    if (n > 1) factors.push_back(n); // The residual factor > sqrt(N)
    return factors;
}
```

Algorithmic Analysis: $\mathcal{O}(\sqrt{N})$ Factorization

Time Complexity: $\mathcal{O}(\sqrt{N})$

- The for loop strictly terminates when $i \cdot i > n$.
- The inner while loop reduces N exponentially, acting as an accelerator.
- In the absolute worst-case scenario (when N is prime), the loop runs exactly $\sqrt{N}$ times.
- For $N = 10^{12}$, maximum operations $\approx 10^6 \implies$ **0.01s** execution time.

Algorithmic Analysis: $\mathcal{O}(\sqrt{N})$ Factorization

Time Complexity: $\mathcal{O}(\sqrt{N})$

- The for loop strictly terminates when $i \cdot i > n$.
- The inner while loop reduces N exponentially, acting as an accelerator.
- In the absolute worst-case scenario (when N is prime), the loop runs exactly $\sqrt{N}$ times.
- For $N = 10^{12}$, maximum operations $\approx 10^6 \implies$ **0.01s** execution time.

Space Complexity: $\mathcal{O}(\log N)$

- The space relies on the vector storing the factors.
- The maximum number of prime factors occurs when N is a power of the smallest prime (2).
- Number of factors $\leq \log_2(N)$.
- For $N = 10^{18}$, the vector holds at most 60 integers.

Example

Task: Trace the code for $N = 44$.

Initial State: $N = 44$, bound ≈ 6.63 . Loop runs for $i \in [2, 6]$.

Example

Task: Trace the code for $N = 44$.

Initial State: $N = 44$, bound ≈ 6.63 . Loop runs for $i \in [2, 6]$.

Execution:

- ① $i = 2$: $44 \pmod{2} == 0 \implies \text{push } 2. N = 22.$
- ② $22 \pmod{2} == 0 \implies \text{push } 2. N = 11.$
- ③ $11 \pmod{2} \neq 0$. Break while.
- ④ $i = 3$: $11 \pmod{3} \neq 0$.
- ⑤ $i = 4, 5, 6$: All fail modulo check. Loop terminates.

Example

Task: Trace the code for $N = 44$.

Initial State: $N = 44$, bound ≈ 6.63 . Loop runs for $i \in [2, 6]$.

Execution:

- ① $i = 2$: $44 \pmod{2} == 0 \implies \text{push } 2$. $N = 22$.
- ② $22 \pmod{2} == 0 \implies \text{push } 2$. $N = 11$.
- ③ $11 \pmod{2} \neq 0$. Break while.
- ④ $i = 3$: $11 \pmod{3} \neq 0$.
- ⑤ $i = 4, 5, 6$: All fail modulo check. Loop terminates.

The Golden Rule Check: Is $N > 1$? Yes, $N = 11$. Push 11. Result: $\{2, 2, 11\}$. (*Notice $11 > \sqrt{44}$, caught flawlessly without looping to 11*).

Precomputation

Precomputation

Scenario: We are given $Q = 10^5$ independent queries. For each query, we must factorize $N \leq 10^7$.

Precomputation

Scenario: We are given $Q = 10^5$ independent queries. For each query, we must factorize $N \leq 10^7$.

Evaluating our $\mathcal{O}(\sqrt{N})$ Algorithm:

$$Q \times \sqrt{N} = 10^5 \times \sqrt{10^7} \approx 3.16 \times 10^8 \text{ operations}$$

Precomputation

Scenario: We are given $Q = 10^5$ independent queries. For each query, we must factorize $N \leq 10^7$.

Evaluating our $\mathcal{O}(\sqrt{N})$ Algorithm:

$$Q \times \sqrt{N} = 10^5 \times \sqrt{10^7} \approx 3.16 \times 10^8 \text{ operations}$$

Question: Flaw

This approach borders on a TLE. Why is this architecture fundamentally flawed for multiple queries? What redundant mathematical work are we performing?

Precomputation

Scenario: We are given $Q = 10^5$ independent queries. For each query, we must factorize $N \leq 10^7$.

Evaluating our $\mathcal{O}(\sqrt{N})$ Algorithm:

$$Q \times \sqrt{N} = 10^5 \times \sqrt{10^7} \approx 3.16 \times 10^8 \text{ operations}$$

Question: Flaw

This approach borders on a TLE. Why is this architecture fundamentally flawed for multiple queries? What redundant mathematical work are we performing?

Conclusion: We are independently verifying the primality of the exact same numbers across different queries. We must transition from *query-time calculation* to *bulk precomputation*.

Optimizing the Sieve

Assume we are building a boolean array to mark composite numbers. We reach the prime $p = 5$. We must mark its multiples as composite: 10, 15, 20, 25, 30...

Optimizing the Sieve

Assume we are building a boolean array to mark composite numbers. We reach the prime $p = 5$. We must mark its multiples as composite: 10, 15, 20, 25, 30...

Question: Identifying Redundancy

Look closely at the first three multiples: 10, 15, and 20. Why is it a strict waste of CPU cycles to mark them when evaluating $p = 5$?

Optimizing the Sieve

Assume we are building a boolean array to mark composite numbers. We reach the prime $p = 5$. We must mark its multiples as composite: 10, 15, 20, 25, 30...

Question: Identifying Redundancy

Look closely at the first three multiples: 10, 15, and 20. Why is it a strict waste of CPU cycles to mark them when evaluating $p = 5$?

The Observation:

- $10 = 2 \times 5$ (Already marked by 2)
- $15 = 3 \times 5$ (Already marked by 3)
- $20 = 2^2 \times 5$ (Already marked by 2)

Sieve of Eratosthenes Formalization

Theorem (Minimal Marking Bound)

When iteratively marking the multiples of a prime $p \in \mathbb{P}$ as composite, the strictly first unmarked multiple is exactly p^2 .

Sieve of Eratosthenes Formalization

Theorem (Minimal Marking Bound)

When iteratively marking the multiples of a prime $p \in \mathbb{P}$ as composite, the strictly first unmarked multiple is exactly p^2 .

Rigorous Justification:

- Let $C = k \cdot p$ represent a composite multiple where $k < p$.
- Because $k > 1$, it possesses a smallest prime factor $q \leq k$.
- Transitively, $q \leq k < p \implies q < p$.
- Therefore, C was already marked during the iteration of the smaller prime q .

Implementing the Sieve

Listing 4: Sieve of Eratosthenes

```
const int MAXN = 1e7;
std::vector<bool> is_prime(MAXN + 1, true);

void sieve() {
    is_prime[0] = is_prime[1] = false;

    for (int i = 2; i * i <= MAXN; i++) {
        if (is_prime[i]) {
            // Optimization applied: Start strictly at i * i
            for (int j = i * i; j <= MAXN; j += i) {
                is_prime[j] = false;
            }
        }
    }
}
```

Algorithmic Analysis: Sieve Execution Time

Question: Harmonic Sums

How many times does the inner loop run? For $p = 2$, it runs $N/2$ times. For $p = 3$, $N/3$ times. What is the sum of this series?

Algorithmic Analysis: Sieve Execution Time

Question: Harmonic Sums

How many times does the inner loop run? For $p = 2$, it runs $N/2$ times. For $p = 3$, $N/3$ times. What is the sum of this series?

Time Complexity Derivation: Let $T(N)$ be the total inner-loop iterations.

$$T(N) = \sum_{p \leq N} \frac{N}{p} = N \sum_{p \leq N} \frac{1}{p}$$

Algorithmic Analysis: Sieve Execution Time

Question: Harmonic Sums

How many times does the inner loop run? For $p = 2$, it runs $N/2$ times. For $p = 3$, $N/3$ times. What is the sum of this series?

Time Complexity Derivation: Let $T(N)$ be the total inner-loop iterations.

$$T(N) = \sum_{p \leq N} \frac{N}{p} = N \sum_{p \leq N} \frac{1}{p}$$

By **Mertens' Second Theorem**, the sum of the reciprocals of primes diverges extremely slowly:

$$\sum_{p \leq N} \frac{1}{p} \approx \ln(\ln N)$$

Algorithmic Analysis: Sieve Execution Time

Question: Harmonic Sums

How many times does the inner loop run? For $p = 2$, it runs $N/2$ times. For $p = 3$, $N/3$ times. What is the sum of this series?

Time Complexity Derivation: Let $T(N)$ be the total inner-loop iterations.

$$T(N) = \sum_{p \leq N} \frac{N}{p} = N \sum_{p \leq N} \frac{1}{p}$$

By **Mertens' Second Theorem**, the sum of the reciprocals of primes diverges extremely slowly:

$$\sum_{p \leq N} \frac{1}{p} \approx \ln(\ln N)$$

Result: Time Complexity is strictly $\mathcal{O}(N \log \log N)$. In CP, this is practically linear.

Algorithmic Analysis: Memory Analysis

The Danger: We solved the Time Limit, but $\mathcal{O}(N)$ space creates a Memory Limit Exceeded (MLE) risk. Standard Codeforces Limit = 256MB.

Algorithmic Analysis: Memory Analysis

The Danger: We solved the Time Limit, but $\mathcal{O}(N)$ space creates a Memory Limit Exceeded (MLE) risk. Standard Codeforces Limit = 256MB.

Assume $N = 10^8$. Let us analyze array types:

- `int is_prime[100000000];`
 - ▶ $10^8 \times 4 \text{ bytes} \approx \mathbf{400 \text{ MB}}$ (Instant MLE Crash)
- `bool is_prime[100000000];`
 - ▶ $10^8 \times 1 \text{ byte} \approx \mathbf{100 \text{ MB}}$ (Safe, but dangerously heavy)
- `std::vector<bool>` or `std::bitset`
 - ▶ C++ physically packs 8 booleans into a single byte.
 - ▶ $10^8/8 \approx \mathbf{12.5 \text{ MB}}$ (Highly Optimized)

Smallest Prime Factor (SPF) Precomputation

Question: Upgrading the Data Structure

A boolean array only tells us *if* N is prime. How can we modify the Sieve array to allow us to extract the exact prime factors of any N in exactly $\mathcal{O}(\log N)$ time?

Smallest Prime Factor (SPF) Precomputation

Question: Upgrading the Data Structure

A boolean array only tells us *if* N is prime. How can we modify the Sieve array to allow us to extract the exact prime factors of any N in exactly $\mathcal{O}(\log N)$ time?

The Logic Shift: Instead of writing `false` when a number is composite, we write the *integer value* of the prime p that crossed it out.

Smallest Prime Factor (SPF) Precomputation

Question: Upgrading the Data Structure

A boolean array only tells us *if* N is prime. How can we modify the Sieve array to allow us to extract the exact prime factors of any N in exactly $\mathcal{O}(\log N)$ time?

The Logic Shift: Instead of writing `false` when a number is composite, we write the *integer value* of the prime p that crossed it out.

Because we iterate p in ascending order $(2, 3, 5 \dots)$, the first prime to cross out C will strictly be its Smallest Prime Factor.

Implementing SPF

Listing 5: SPF Precomputation

```
const int MAXN = 1e7;
int spf[MAXN + 1];

void sieveSPF() {
    for (int i = 2; i <= MAXN; i++) spf[i] = i; // Base case

    for (int i = 2; i * i <= MAXN; i++) {
        if (spf[i] == i) { // If i is prime
            for (int j = i * i; j <= MAXN; j += i) {
                // Mark ONLY if it hasn't been marked by a smaller prime
                if (spf[j] == j) spf[j] = i;
            }
        }
    }
}
```

Precomputation Complexity:

- **Time:** $\mathcal{O}(N \log \log N)$ (Identical to standard Sieve).
- **Space:** $\mathcal{O}(N)$. Because we must store integers, we cannot use bit-packing. `int spf[1e7]` requires $\approx 40\text{MB}$, safely within limits.

Precomputation Complexity:

- **Time:** $\mathcal{O}(N \log \log N)$ (Identical to standard Sieve).
- **Space:** $\mathcal{O}(N)$. Because we must store integers, we cannot use bit-packing. `int spf[1e7]` requires $\approx 40\text{MB}$, safely within limits.

Query Processing Complexity: $\mathcal{O}(\log N)$

- To factorize N , we do $N = N/\text{spf}[N]$ repeatedly.
- In the worst-case (powers of 2), we divide by 2 at each step.
- Total divisions per query: $\log_2 N$.
- 10^5 queries on $N = 10^7$ now takes $\approx 10^5 \times 24 \approx$

Example

Task: Given a precomputed spf array, extract factors for $N = 120$.

Example

Task: Given a precomputed `spf` array, extract factors for $N = 120$.

Execution:

- $N = 120 \implies \text{spf}[120] = 2 \implies \text{record } 2. \quad (120/2 = 60)$
- $N = 60 \implies \text{spf}[60] = 2 \implies \text{record } 2. \quad (60/2 = 30)$
- $N = 30 \implies \text{spf}[30] = 2 \implies \text{record } 2. \quad (30/2 = 15)$
- $N = 15 \implies \text{spf}[15] = 3 \implies \text{record } 3. \quad (15/3 = 5)$
- $N = 5 \implies \text{spf}[5] = 5 \implies \text{record } 5. \quad (5/5 = 1)$

Result: $\{2, 2, 2, 3, 5\}$.

Advanced Modular Arithmetic

Standard arithmetic operations $(+, -, \times)$ distribute natively over modulo M .

Advanced Modular Arithmetic

Standard arithmetic operations $(+, -, \times)$ distribute natively over modulo M .

Question: The Division Failure

Why is the following mathematical statement strictly false?

$$(A/B) \pmod{M} = (A \pmod{M} / B \pmod{M}) \pmod{M}$$

Provide a concrete counter-example.

Advanced Modular Arithmetic

Standard arithmetic operations $(+, -, \times)$ distribute natively over modulo M .

Question: The Division Failure

Why is the following mathematical statement strictly false?

$$(A/B) \pmod{M} = (A \pmod{M} / B \pmod{M}) \pmod{M}$$

Provide a concrete counter-example.

Counter-Example: $(12/4) \pmod{5}$.

- *Mathematically Correct:* $(12/4) = 3 \implies 3 \pmod{5} = \mathbf{3}$.
- *Distributive Trap:* $(12 \pmod{5}) / (4 \pmod{5}) = 2/4 = \mathbf{0.5}$. (Invalid in $\mathbb{Z}$)

Modular Inverse

Since standard division fails, we must bypass the division operator entirely. Dividing by B is algebraically identical to multiplying by its inverse B^{-1} .

$$B \cdot B^{-1} \equiv 1 \pmod{M}$$

Modular Inverse

Since standard division fails, we must bypass the division operator entirely. Dividing by B is algebraically identical to multiplying by its inverse B^{-1} .

$$B \cdot B^{-1} \equiv 1 \pmod{M}$$

Fermat's Little Theorem (FLT): Let $p \in \mathbb{P}$. If $\gcd(a, p) = 1$, then:

$$a^{p-1} \equiv 1 \pmod{p}$$

Question: Extracting the Inverse

Given Fermat's equation above, how can we algebraically isolate a^{-1} ?

Extracting the Inverse (Resolution)

Derivation: Multiply both sides of FLT by a^{-1} .

$$\begin{aligned} a^{p-1} \cdot a^{-1} &\equiv 1 \cdot a^{-1} \pmod{p} \\ a^{p-2} &\equiv a^{-1} \pmod{p} \end{aligned}$$

The CP Takeaway: Whenever a problem asks you to calculate $\frac{A}{B} \pmod{P}$ (where P is prime, like $10^9 + 7$), you calculate:

$$(A \cdot B^{P-2}) \pmod{P}$$

Implementing Modular Inverse

To compute $B^{P-2} \pmod{P}$, we must use Binary Exponentiation.

Listing 6: $\mathcal{O}(\log P)$ Modular Inverse

```
long long power(long long base, long long exp, long long mod) {
    long long res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % mod;
        base = (base * base) % mod;
        exp /= 2;
    }
    return res;
}

long long modInverse(long long n, long long p) {
    return power(n, p - 2, p); // ONLY VALID IF P IS PRIME
}
```

Time Complexity: $\mathcal{O}(\log P)$

- The `while` loop halves the exponent at every iteration (`exp /= 2`).
- Therefore, the loop runs exactly $\log_2(\text{exp})$ times.
- For $P = 10^9 + 7$, $\text{exp} = 10^9 + 5$.
- $\log_2(10^9) \approx 30$ iterations. Execution is strictly $\mathcal{O}(1)$ practically.

Algorithmic Analysis: Modular Inverse

Time Complexity: $\mathcal{O}(\log P)$

- The `while` loop halves the exponent at every iteration (`exp /= 2`).
- Therefore, the loop runs exactly $\log_2(\text{exp})$ times.
- For $P = 10^9 + 7$, $\text{exp} = 10^9 + 5$.
- $\log_2(10^9) \approx 30$ iterations. Execution is strictly $\mathcal{O}(1)$ practically.

Space Complexity: $\mathcal{O}(1)$

- The algorithm updates the values `res`, `base`, and `exp` iteratively in-place.
- No arrays or recursive call stacks are generated. Memory overhead is strictly 24 bytes (three `long long` variables).

Example

Scenario: We are operating modulo $10^9 + 7$. We need to calculate $(50/2) \pmod{10^9 + 7}$.

Example

Scenario: We are operating modulo $10^9 + 7$. We need to calculate $(50/2) \pmod{10^9 + 7}$.

Step 1: Rewrite as Multiplication

$$50 \cdot 2^{-1} \pmod{10^9 + 7}$$

Example

Scenario: We are operating modulo $10^9 + 7$. We need to calculate $(50/2) \pmod{10^9 + 7}$.

Step 1: Rewrite as Multiplication

$$50 \cdot 2^{-1} \pmod{10^9 + 7}$$

Step 2: Apply FLT to find 2^{-1}

$$2^{-1} \equiv 2^{(10^9+7)-2} \pmod{10^9 + 7}$$

$$2^{-1} \equiv 2^{10^9+5} \pmod{10^9 + 7}$$

Example

Scenario: We are operating modulo $10^9 + 7$. We need to calculate $(50/2) \pmod{10^9 + 7}$.

Step 1: Rewrite as Multiplication

$$50 \cdot 2^{-1} \pmod{10^9 + 7}$$

Step 2: Apply FLT to find 2^{-1}

$$2^{-1} \equiv 2^{(10^9+7)-2} \pmod{10^9 + 7}$$

$$2^{-1} \equiv 2^{10^9+5} \pmod{10^9 + 7}$$

Step 3: Execution Feed base = 2, exp = 1000000005, mod = 1000000007 into the Binary Exponentiation function. Multiply the returned result by 50 and apply modulo.

Euler's Totient Function (ϕ)

Fermat's Little Theorem is powerful, but it completely breaks if the modulo N is composite.

Euler's Totient Function (ϕ)

Fermat's Little Theorem is powerful, but it completely breaks if the modulo N is composite.

Definition

$\phi(N)$ outputs the exact count of integers strictly up to N that are coprime to N .

$$\phi(N) = |\{k \in \mathbb{Z} : 1 \leq k \leq N \wedge \gcd(k, N) = 1\}|$$

Euler's Totient Function (ϕ)

Fermat's Little Theorem is powerful, but it completely breaks if the modulo N is composite.

Definition

$\phi(N)$ outputs the exact count of integers strictly up to N that are coprime to N .

$$\phi(N) = |\{k \in \mathbb{Z} : 1 \leq k \leq N \wedge \gcd(k, N) = 1\}|$$

Theorem (Euler's Theorem)

If $\gcd(a, N) = 1$:

$$a^{\phi(N)} \equiv 1 \pmod{N} \implies a^{-1} \equiv a^{\phi(N)-1} \pmod{N}$$

Question: Structural Reduction

If N happens to be a prime number, what does $\phi(N)$ evaluate to?

Proof: Euler's Theorem (Part 1)

Proof: Euler's Theorem (Part 1)

Goal: Prove $a^{\phi(N)} \equiv 1 \pmod{N}$ given $\gcd(a, N) = 1$.

Proof: Euler's Theorem (Part 1)

Goal: Prove $a^{\phi(N)} \equiv 1 \pmod{N}$ given $\gcd(a, N) = 1$.

Step 1: The Coprime Set Let S be the set of all integers strictly less than N that are coprime to N . By definition, there are exactly $\phi(N)$ elements in this set.

$$S = \{x_1, x_2, \dots, x_{\phi(N)}\}$$

Proof: Euler's Theorem (Part 1)

Goal: Prove $a^{\phi(N)} \equiv 1 \pmod{N}$ given $\gcd(a, N) = 1$.

Step 1: The Coprime Set Let S be the set of all integers strictly less than N that are coprime to N . By definition, there are exactly $\phi(N)$ elements in this set.

$$S = \{x_1, x_2, \dots, x_{\phi(N)}\}$$

Step 2: The Multiplication Mapping Multiply every single element in S by a to form a new set S' :

$$S' = \{ax_1, ax_2, \dots, ax_{\phi(N)}\} \pmod{N}$$

Proof: Euler's Theorem (Part 1)

Goal: Prove $a^{\phi(N)} \equiv 1 \pmod{N}$ given $\gcd(a, N) = 1$.

Step 1: The Coprime Set Let S be the set of all integers strictly less than N that are coprime to N . By definition, there are exactly $\phi(N)$ elements in this set.

$$S = \{x_1, x_2, \dots, x_{\phi(N)}\}$$

Step 2: The Multiplication Mapping Multiply every single element in S by a to form a new set S' :

$$S' = \{ax_1, ax_2, \dots, ax_{\phi(N)}\} \pmod{N}$$

Question: The Permutation Logic

If you take modulo N of every element in S' , why must S' contain the *exact same numbers* as S , just in a scrambled order?

Proof: Euler's Theorem (Part 2)

Proof: Euler's Theorem (Part 2)

The Resolution (Why it Permutes): Because $\gcd(a, N) = 1$, multiplying by a preserves coprimality. Furthermore, no two elements collide ($ax_i \equiv ax_j \implies x_i \equiv x_j$). Thus, S' is just a perfect permutation of S .

Proof: Euler's Theorem (Part 2)

The Resolution (Why it Permutes): Because $\gcd(a, N) = 1$, multiplying by a preserves coprimality. Furthermore, no two elements collide ($ax_i \equiv ax_j \implies x_i \equiv x_j$). Thus, S' is just a perfect permutation of S .

Step 3: Equating the Products Since S and S' contain the exact same numbers modulo N , the product of their elements must be strictly congruent:

$$\prod_{i=1}^{\phi(N)} (a \cdot x_i) \equiv \prod_{i=1}^{\phi(N)} x_i \pmod{N}$$

Proof: Euler's Theorem (Part 3)

Proof: Euler's Theorem (Part 3)

Step 4: Algebraic Extraction Factor out the a 's. Since there are exactly $\phi(N)$ terms, we extract $a^{\phi(N)}$:

$$a^{\phi(N)} \prod_{i=1}^{\phi(N)} x_i \equiv 1 \cdot \prod_{i=1}^{\phi(N)} x_i \pmod{N}$$

Because every x_i is coprime to N , we can safely divide both sides by the product, leaving us with:

$$a^{\phi(N)} \equiv 1 \pmod{N} \quad \blacksquare$$

Computing Totient Function

Computing Totient Function

Mathematical Calculation: For a prime factorization $N = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_k^{\alpha_k}$:

Computing Totient Function

Mathematical Calculation: For a prime factorization $N = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_k^{\alpha_k}$:

$$\phi(N) = N \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right)$$

Computing Totient Function

Mathematical Calculation: For a prime factorization $N = p_1^{\alpha_1} p_2^{\alpha_2} \dots p_k^{\alpha_k}$:

$$\phi(N) = N \prod_{i=1}^k \left(1 - \frac{1}{p_i}\right)$$

Listing 10: $\mathcal{O}(\sqrt{N})$ ETF Computation

```
long long getPhi(long long n) {
    long long res = n;
    for (long long i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            res -= res / i; // Equivalent to res * (1 - 1/i)
        }
    }
    if (n > 1) res -= res / n;
    return res;
}
```

Algorithmic Analysis: Euler's Totient Function

Time Complexity: $\mathcal{O}(\sqrt{N})$

- The implementation is fundamentally identical to the $\mathcal{O}(\sqrt{N})$ Prime Factorization code.
- We iterate up to $\sqrt{N}$ and strip prime factors dynamically.
- The calculation `res -= res / i` occurs inside the factorization loop, adding only $\mathcal{O}(1)$ constant time overhead per unique prime.

Algorithmic Analysis: Euler's Totient Function

Time Complexity: $\mathcal{O}(\sqrt{N})$

- The implementation is fundamentally identical to the $\mathcal{O}(\sqrt{N})$ Prime Factorization code.
- We iterate up to $\sqrt{N}$ and strip prime factors dynamically.
- The calculation `res -= res / i` occurs inside the factorization loop, adding only $\mathcal{O}(1)$ constant time overhead per unique prime.

Space Complexity: $\mathcal{O}(1)$

- Unlike standard factorization where we stored factors in a vector, here we merely apply the mathematical fraction *on the fly*.
- Because nothing is stored, memory drops from $\mathcal{O}(\log N)$ to $\mathcal{O}(1)$.

Example

Task: Compute $\phi(12)$ manually, then mathematically verify with the formula.

Example

Task: Compute $\phi(12)$ manually, then mathematically verify with the formula.

Manual Verification: Integers ≤ 12 : $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$.

- Set of integers coprime to 12: $\{1, 5, 7, 11\}$.
- Cardinality count: 4.

Example

Task: Compute $\phi(12)$ manually, then mathematically verify with the formula.

Manual Verification: Integers ≤ 12 : $\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12\}$.

- Set of integers coprime to 12: $\{1, 5, 7, 11\}$.
- Cardinality count: 4.

Formula Verification: Prime factorization of $12 = 2^2 \cdot 3^1$.

$$\phi(12) = 12 \left(1 - \frac{1}{2}\right) \left(1 - \frac{1}{3}\right)$$

$$\phi(12) = 12 \left(\frac{1}{2}\right) \left(\frac{2}{3}\right) = 4.$$

Lecture Summary: The "Why"

Why does precomputation exist?

- Because independent queries duplicate work. The Sieve transforms query time from $\mathcal{O}(\sqrt{N})$ to $\mathcal{O}(1)$ or $\mathcal{O}(\log N)$.

Why do we need space optimization?

- Time limits and Memory limits are inversely related. $\mathcal{O}(N)$ memory on massive arrays triggers MLE. Bit-packing solves this.

Why do prime modulus matter?

- Standard division fails modular distribution. Primes unlock Fermat's Little Theorem, allowing multiplication by an inverse in $\mathcal{O}(\log P)$ time.

Problem 1: T-primes (CF 230B)

Problem Statement: A number is called a "T-prime" if it has **exactly three** distinct positive divisors. You are given $Q = 10^5$ queries, each providing a number $N \leq 10^{12}$. Output YES if N is a T-prime.

Problem 1: T-primes (CF 230B)

Problem Statement: A number is called a "T-prime" if it has **exactly three** distinct positive divisors. You are given $Q = 10^5$ queries, each providing a number $N \leq 10^{12}$. Output YES if N is a T-prime.

The Naive Trap

If you run our $O(\sqrt{N})$ divisor counting algorithm for 10^5 queries where $N = 10^{12}$, operations $\approx 10^5 \times 10^6 = 10^{11}$. **Strict TLE.**

Problem 1: T-primes (CF 230B)

Problem Statement: A number is called a "T-prime" if it has **exactly three** distinct positive divisors. You are given $Q = 10^5$ queries, each providing a number $N \leq 10^{12}$. Output YES if N is a T-prime.

The Naive Trap

If you run our $O(\sqrt{N})$ divisor counting algorithm for 10^5 queries where $N = 10^{12}$, operations $\approx 10^5 \times 10^6 = 10^{11}$. **Strict TLE.**

The Mathematical Resolution:

- What type of number has exactly 3 divisors?

Problem 1: T-primes (CF 230B)

Problem Statement: A number is called a "T-prime" if it has **exactly three** distinct positive divisors. You are given $Q = 10^5$ queries, each providing a number $N \leq 10^{12}$. Output YES if N is a T-prime.

The Naive Trap

If you run our $O(\sqrt{N})$ divisor counting algorithm for 10^5 queries where $N = 10^{12}$, operations $\approx 10^5 \times 10^6 = 10^{11}$. **Strict TLE.**

The Mathematical Resolution:

- What type of number has exactly 3 divisors?
- It must be the **square of a prime number** p . Divisors: $\{1, p, p^2\}$.

Problem 1: T-primes (CF 230B)

Problem Statement: A number is called a "T-prime" if it has **exactly three** distinct positive divisors. You are given $Q = 10^5$ queries, each providing a number $N \leq 10^{12}$. Output YES if N is a T-prime.

The Naive Trap

If you run our $O(\sqrt{N})$ divisor counting algorithm for 10^5 queries where $N = 10^{12}$, operations $\approx 10^5 \times 10^6 = 10^{11}$. **Strict TLE.**

The Mathematical Resolution:

- What type of number has exactly 3 divisors?
- It must be the **square of a prime number** p . Divisors: $\{1, p, p^2\}$.
- **Algorithm:** Precompute a Sieve up to $\sqrt{10^{12}} = 10^6$. For each query N , check if $\sqrt{N}$ is an integer, and if that integer is marked prime in your Sieve! Time complexity drops to $\mathcal{O}(1)$ per query.

Problem 2: Sherlock (CF 776B)

Problem Statement: You need to color integers from 2 to $N + 1$. If integer A is a prime factor of integer B , they **must** have different colors. Find the minimum number of colors needed. ($N \leq 10^5$).

Problem 2: Sherlock (CF 776B)

Problem Statement: You need to color integers from 2 to $N + 1$. If integer A is a prime factor of integer B , they **must** have different colors. Find the minimum number of colors needed. ($N \leq 10^5$).

The Graph Theory Trap

Many students see "colors" and "factors" and immediately try to build a massive graph to run DFS coloring. This is massive overkill and risks TLE/MLE.

Problem 2: Sherlock (CF 776B)

Problem Statement: You need to color integers from 2 to $N + 1$. If integer A is a prime factor of integer B , they **must** have different colors. Find the minimum number of colors needed. ($N \leq 10^5$).

The Graph Theory Trap

Many students see "colors" and "factors" and immediately try to build a massive graph to run DFS coloring. This is massive overkill and risks TLE/MLE.

The Structural Resolution:

Problem 2: Sherlock (CF 776B)

Problem Statement: You need to color integers from 2 to $N + 1$. If integer A is a prime factor of integer B , they **must** have different colors. Find the minimum number of colors needed. ($N \leq 10^5$).

The Graph Theory Trap

Many students see "colors" and "factors" and immediately try to build a massive graph to run DFS coloring. This is massive overkill and risks TLE/MLE.

The Structural Resolution:

- We only ever need **2 colors** (if $N > 2$).

Problem 2: Sherlock (CF 776B)

Problem Statement: You need to color integers from 2 to $N + 1$. If integer A is a prime factor of integer B , they **must** have different colors. Find the minimum number of colors needed. ($N \leq 10^5$).

The Graph Theory Trap

Many students see "colors" and "factors" and immediately try to build a massive graph to run DFS coloring. This is massive overkill and risks TLE/MLE.

The Structural Resolution:

- We only ever need **2 colors** (if $N > 2$).
- **Color 1:** All prime numbers.

Problem 2: Sherlock (CF 776B)

Problem Statement: You need to color integers from 2 to $N + 1$. If integer A is a prime factor of integer B , they **must** have different colors. Find the minimum number of colors needed. ($N \leq 10^5$).

The Graph Theory Trap

Many students see "colors" and "factors" and immediately try to build a massive graph to run DFS coloring. This is massive overkill and risks TLE/MLE.

The Structural Resolution:

- We only ever need **2 colors** (if $N > 2$).
- **Color 1:** All prime numbers.
- **Color 2:** All composite numbers.

Problem 2: Sherlock (CF 776B)

Problem Statement: You need to color integers from 2 to $N + 1$. If integer A is a prime factor of integer B , they **must** have different colors. Find the minimum number of colors needed. ($N \leq 10^5$).

The Graph Theory Trap

Many students see "colors" and "factors" and immediately try to build a massive graph to run DFS coloring. This is massive overkill and risks TLE/MLE.

The Structural Resolution:

- We only ever need **2 colors** (if $N > 2$).
- **Color 1:** All prime numbers.
- **Color 2:** All composite numbers.
- *Proof:* A prime number can never be a prime factor of another prime number. A composite number's prime factors are strictly primes (Color 1). Therefore, no two numbers of the same color share a prime-factor relationship. Run the Sieve, print 1 for primes, 2 for composites!

Problem 3: Exponentiation II (CSES 1712)

Problem Statement: Given three integers A , B , and C , calculate $A^{B^C} \pmod{10^9 + 7}$.
Constraints: $A, B, C \leq 10^9$.

Problem 3: Exponentiation II (CSES 1712)

Problem Statement: Given three integers A , B , and C , calculate $A^{B^C} \pmod{10^9 + 7}$.
Constraints: $A, B, C \leq 10^9$.

The Modulo Distribution Trap

Students immediately write:

```
long long powerBC = power(B, C, 1000000007);  
long long ans = power(A, powerBC, 1000000007);
```

This gives a **Wrong Answer (WA)**. Modulo does **not** distribute to exponents!

Problem 3: Exponentiation II (CSES 1712)

Problem Statement: Given three integers A, B , and C , calculate $A^{B^C} \pmod{10^9 + 7}$.

Constraints: $A, B, C \leq 10^9$.

The Modulo Distribution Trap

Students immediately write:

```
long long powerBC = power(B, C, 1000000007);  
long long ans = power(A, powerBC, 1000000007);
```

This gives a **Wrong Answer (WA)**. Modulo does **not** distribute to exponents!

The FLT / ETF Resolution:

- By Fermat's Little Theorem / Euler's Totient Theorem, the exponent cycles every $\phi(M)$ times.
- Since $M = 10^9 + 7$ is prime, $\phi(M) = 10^9 + 6$.
- **Correct Algorithm:** First, compute $P = B^C \pmod{10^9 + 6}$. Then compute $A^P \pmod{10^9 + 7}$.

Problem 4: Same GCDs (CF 1295D)

Problem Statement: You are given two integers a and m ($1 \leq a < m \leq 10^{13}$). Calculate the number of integers x such that $0 \leq x < m$ and:

$$\gcd(a, m) = \gcd(a + x, m)$$

Problem 4: Same GCDs (CF 1295D)

Problem Statement: You are given two integers a and m ($1 \leq a < m \leq 10^{13}$). Calculate the number of integers x such that $0 \leq x < m$ and:

$$\gcd(a, m) = \gcd(a + x, m)$$

The Bruteforce Trap

A naive solution would loop x from 0 to $m - 1$ and check the GCD condition. Since $m \leq 10^{13}$, and there are up to 50 test cases, operations $\approx 5 \times 10^{14}$. **Result: Immediate Time Limit Exceeded (TLE).**

Problem 4: Same GCDs (CF 1295D)

Problem Statement: You are given two integers a and m ($1 \leq a < m \leq 10^{13}$). Calculate the number of integers x such that $0 \leq x < m$ and:

$$\gcd(a, m) = \gcd(a + x, m)$$

The Bruteforce Trap

A naive solution would loop x from 0 to $m - 1$ and check the GCD condition. Since $m \leq 10^{13}$, and there are up to 50 test cases, operations $\approx 5 \times 10^{14}$. **Result: Immediate Time Limit Exceeded (TLE).**

The Goal: We must solve this in exactly $\mathcal{O}(\sqrt{m})$ time. How can we mathematically break down the equation?

Factoring the GCD

Let $g = \gcd(a, m)$. We want to find x such that:

$$\gcd(a + x, m) = g$$

Factoring the GCD

Let $g = \gcd(a, m)$. We want to find x such that:

$$\gcd(a + x, m) = g$$

Step 1: Divide by the GCD By the fundamental properties of the GCD, we can divide both sides by g :

$$\gcd\left(\frac{a + x}{g}, \frac{m}{g}\right) = 1$$

Factoring the GCD

Let $g = \gcd(a, m)$. We want to find x such that:

$$\gcd(a + x, m) = g$$

Step 1: Divide by the GCD By the fundamental properties of the GCD, we can divide both sides by g :

$$\gcd\left(\frac{a + x}{g}, \frac{m}{g}\right) = 1$$

Step 2: Analyzing the Range Let $k = \frac{a+x}{g}$. We need k to be coprime to $\frac{m}{g}$.

Factoring the GCD

Let $g = \gcd(a, m)$. We want to find x such that:

$$\gcd(a + x, m) = g$$

Step 1: Divide by the GCD By the fundamental properties of the GCD, we can divide both sides by g :

$$\gcd\left(\frac{a+x}{g}, \frac{m}{g}\right) = 1$$

Step 2: Analyzing the Range Let $k = \frac{a+x}{g}$. We need k to be coprime to $\frac{m}{g}$.

Question: The Upper and Lower Bounds

If x strictly ranges from 0 to $m - 1$, what is the exact range of k ?

Step 3: The Interval Length When $x = 0$, the minimum value is $k = \frac{a}{g}$.
When $x = m - 1$, the maximum value is $k = \frac{a+m-1}{g}$.

Periodic Coprimality

Step 3: The Interval Length When $x = 0$, the minimum value is $k = \frac{a}{g}$.

When $x = m - 1$, the maximum value is $k = \frac{a+m-1}{g}$.

The length of this interval is exactly:

$$\frac{a + m - 1}{g} - \frac{a}{g} + 1 = \frac{m}{g}$$

Periodic Coprimality

Step 3: The Interval Length When $x = 0$, the minimum value is $k = \frac{a}{g}$.

When $x = m - 1$, the maximum value is $k = \frac{a+m-1}{g}$.

The length of this interval is exactly:

$$\frac{a + m - 1}{g} - \frac{a}{g} + 1 = \frac{m}{g}$$

The Final Insight: We are looking for the count of numbers in an interval of exact length $\frac{m}{g}$ that are coprime to $\frac{m}{g}$.

Periodic Coprimality

Step 3: The Interval Length When $x = 0$, the minimum value is $k = \frac{a}{g}$.

When $x = m - 1$, the maximum value is $k = \frac{a+m-1}{g}$.

The length of this interval is exactly:

$$\frac{a + m - 1}{g} - \frac{a}{g} + 1 = \frac{m}{g}$$

The Final Insight: We are looking for the count of numbers in an interval of exact length $\frac{m}{g}$ that are coprime to $\frac{m}{g}$.

Because coprimality is periodic, the number of coprimes in *any* contiguous interval of length K is exactly $\phi(K)$.

Solution: The answer is simply $\phi\left(\frac{m}{\gcd(a,m)}\right)!$

Implementation: Same GCDs

Since $m \leq 10^{13}$, we cannot use a Sieve. We must use our $\mathcal{O}(\sqrt{N})$ ETF function.

Implementation: Same GCDs

Listing 11: $\mathcal{O}(\sqrt{M})$ Solution for CF 1295D

```
long long gcd(long long a, long long b) {  
    return b == 0 ? a : gcd(b, a % b);  
}  
  
// O(sqrt(N)) Euler's Totient Function  
long long getPhi(long long n) {  
    long long res = n;  
    for (long long i = 2; i * i <= n; i++) {  
        if (n % i == 0) {  
            while (n % i == 0) n /= i;  
            res -= res / i;  
        }  
    }  
    if (n > 1) res -= res / n;  
    return res;  
}
```

Implementation: Same GCDs

```
void solve() {  
    long long a, m;  
    cin >> a >> m;  
    long long g = gcd(a, m);  
    cout << getPhi(m / g) << "\n";  
}
```

Problem 5: The Number of Pairs (CF 1499D)

Problem Statement: Given integers c, d , and x ($1 \leq c, d, x \leq 2 \cdot 10^7$). Find the total number of integer pairs (a, b) satisfying the following equation:

$$c \cdot \text{lcm}(a, b) - d \cdot \text{gcd}(a, b) = x$$

You must process up to $Q = 10^4$ test cases within a strict 2.0-second time limit.

Problem 5: The Number of Pairs (CF 1499D)

Problem Statement: Given integers c, d , and x ($1 \leq c, d, x \leq 2 \cdot 10^7$). Find the total number of integer pairs (a, b) satisfying the following equation:

$$c \cdot \text{lcm}(a, b) - d \cdot \text{gcd}(a, b) = x$$

You must process up to $Q = 10^4$ test cases within a strict 2.0-second time limit.

The Linear Search Trap

Iterating over all candidate pairs (a, b) directly is completely impossible since a, b can grow arbitrarily large. We must reduce the equation using structural divisibility properties.

Problem 5: The Number of Pairs (CF 1499D)

Problem Statement: Given integers c, d , and x ($1 \leq c, d, x \leq 2 \cdot 10^7$). Find the total number of integer pairs (a, b) satisfying the following equation:

$$c \cdot \text{lcm}(a, b) - d \cdot \text{gcd}(a, b) = x$$

You must process up to $Q = 10^4$ test cases within a strict 2.0-second time limit.

The Linear Search Trap

Iterating over all candidate pairs (a, b) directly is completely impossible since a, b can grow arbitrarily large. We must reduce the equation using structural divisibility properties.

Question: How can we factor out the Greatest Common Divisor to isolate coprimality?

Structural Equation Reduction

Let $g = \gcd(a, b)$. We can cleanly parameterize a and b using coprime kernels:

$$a = g \cdot A, \quad b = g \cdot B \quad \text{where} \quad \gcd(A, B) = 1$$

Structural Equation Reduction

Let $g = \gcd(a, b)$. We can cleanly parameterize a and b using coprime kernels:

$$a = g \cdot A, \quad b = g \cdot B \quad \text{where} \quad \gcd(A, B) = 1$$

Substitute these parameters into our standard LCM relation $\text{lcm}(a, b) = g \cdot A \cdot B$:

$$c \cdot (g \cdot A \cdot B) - d \cdot g = x \implies g \cdot (c \cdot A \cdot B - d) = x$$

Structural Equation Reduction

Let $g = \gcd(a, b)$. We can cleanly parameterize a and b using coprime kernels:

$$a = g \cdot A, \quad b = g \cdot B \quad \text{where} \quad \gcd(A, B) = 1$$

Substitute these parameters into our standard LCM relation $\text{lcm}(a, b) = g \cdot A \cdot B$:

$$c \cdot (g \cdot A \cdot B) - d \cdot g = x \implies g \cdot (c \cdot A \cdot B - d) = x$$

The Divisor Criterion: This structural form reveals that g **must** be a perfectly clean divisor of x .

$$c \cdot A \cdot B = \frac{x}{g} + d \implies A \cdot B = \frac{\frac{x}{g} + d}{c}$$

Structural Equation Reduction

Let $g = \gcd(a, b)$. We can cleanly parameterize a and b using coprime kernels:

$$a = g \cdot A, \quad b = g \cdot B \quad \text{where} \quad \gcd(A, B) = 1$$

Substitute these parameters into our standard LCM relation $\text{lcm}(a, b) = g \cdot A \cdot B$:

$$c \cdot (g \cdot A \cdot B) - d \cdot g = x \implies g \cdot (c \cdot A \cdot B - d) = x$$

The Divisor Criterion: This structural form reveals that g **must** be a perfectly clean divisor of x .

$$c \cdot A \cdot B = \frac{x}{g} + d \implies A \cdot B = \frac{\frac{x}{g} + d}{c}$$

Algorithmic Shift: Loop over all valid divisors g of x (in $\mathcal{O}(\sqrt{x})$). For each g , if $K = \frac{x/g+d}{c}$ is an integer, our target shifts to finding the number of coprime pairs (A, B) such that $A \cdot B = K$.

The Prime Combinatorics Step

We need to count pairs (A, B) such that $A \cdot B = K$ under the strict condition $\gcd(A, B) = 1$.

The Prime Combinatorics Step

We need to count pairs (A, B) such that $A \cdot B = K$ under the strict condition $\gcd(A, B) = 1$.

Consider the unique prime factorization of our target constant K :

$$K = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdots p_k^{\alpha_k}$$

The Prime Combinatorics Step

We need to count pairs (A, B) such that $A \cdot B = K$ under the strict condition $\gcd(A, B) = 1$.

Consider the unique prime factorization of our target constant K :

$$K = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdots p_k^{\alpha_k}$$

The Coprimality Constraint:

- For any prime p_i , its entire prime-power component $p_i^{\alpha_i}$ **cannot** be split between A and B . If it were split, both A and B would share the factor p_i , violating $\gcd(A, B) = 1$.
- Therefore, for each unique prime factor p_i , we face a binary decision: assign $p_i^{\alpha_i}$ entirely to A , or entirely to B .

The Prime Combinatorics Step

We need to count pairs (A, B) such that $A \cdot B = K$ under the strict condition $\gcd(A, B) = 1$.

Consider the unique prime factorization of our target constant K :

$$K = p_1^{\alpha_1} \cdot p_2^{\alpha_2} \cdots p_k^{\alpha_k}$$

The Coprimality Constraint:

- For any prime p_i , its entire prime-power component $p_i^{\alpha_i}$ **cannot** be split between A and B . If it were split, both A and B would share the factor p_i , violating $\gcd(A, B) = 1$.
- Therefore, for each unique prime factor p_i , we face a binary decision: assign $p_i^{\alpha_i}$ entirely to A , or entirely to B .

Conclusion: If K has $\omega(K)$ unique prime factors, there are exactly $2^{\omega(K)}$ valid pairs!

Precomputing Prime Counts via Sieve

Answering each query requires looping over the divisors of x . For each valid divisor, we must find $\omega(K)$ in $\mathcal{O}(1)$ to completely avoid a query-time TLE.

Precomputing Prime Counts via Sieve

Answering each query requires looping over the divisors of x . For each valid divisor, we must find $\omega(K)$ in $\mathcal{O}(1)$ to completely avoid a query-time TLE.

The Structural Sieve Extension: We can modify our classical Sieve of Eratosthenes to count the unique prime factors (ω) for every integer up to the maximum possible boundary of $K \approx 2 \cdot 10^7$:

Precomputing Prime Counts via Sieve

Answering each query requires looping over the divisors of x . For each valid divisor, we must find $\omega(K)$ in $\mathcal{O}(1)$ to completely avoid a query-time TLE.

The Structural Sieve Extension: We can modify our classical Sieve of Eratosthenes to count the unique prime factors (ω) for every integer up to the maximum possible boundary of $K \approx 2 \cdot 10^7$:

Instead of maintaining a boolean flag, we maintain an integer array. If we find an untouched value $i == 0$, it is strictly a prime. We step through all its multiples and increment their unique factor counters by 1.

Total Precomputation Time: $\mathcal{O}(MAX \log \log MAX)$

Implementation (Precomputation)

Listing 12: Sieve Precomputation for CF 1499D

```
const int MAXV = 2e7;
int omega[MAXV + 1];

void precompute() {
    // Modified Sieve to count distinct prime factors in bulk
    for (int i = 2; i <= MAXV; i++) {
        if (omega[i] == 0) { // i is prime
            for (int j = i; j <= MAXV; j += i) {
                omega[j]++;
            }
        }
    }
}

// O(1) query to get 2^(omega(k))
long long getPairs(long long k) {
    if (k < 1 || k > MAXV) return 0;
    return 1LL << omega[k]; // Bitshift is equivalent to 2^x
}
```

Implementation (Query Processing)

Listing 13: $O(\sqrt{x})$ Query Processing

```
void solve() {
    long long c, d, x;
    cin >> c >> d >> x;
    long long total_pairs = 0;

    // Loop over all divisors of x
    for (long long g = 1; g * g <= x; g++) {
        if (x % g == 0) {
            // Case 1: Divisor is g
            long long val1 = x / g + d;
            if (val1 % c == 0) total_pairs += getPairs(val1 / c);

            // Case 2: Divisor is x / g
            if (g * g != x) {
                long long val2 = g + d;
                if (val2 % c == 0) total_pairs += getPairs(val2 / c);
            }
        }
    }
    cout << total_pairs << "\n";
}
```